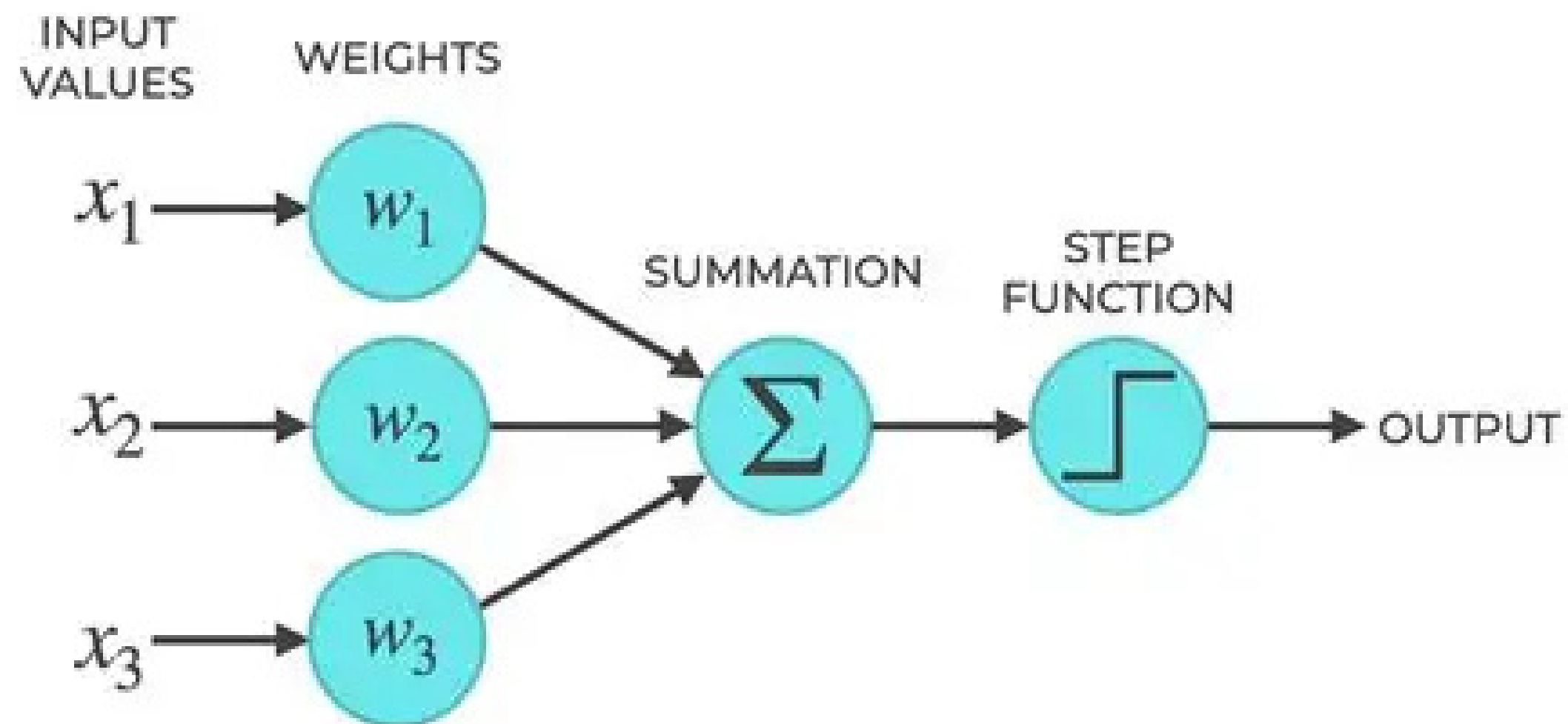


PERCEPTRONS, EXPLAINED



A **Perceptron** is an **Artificial Neuron**.

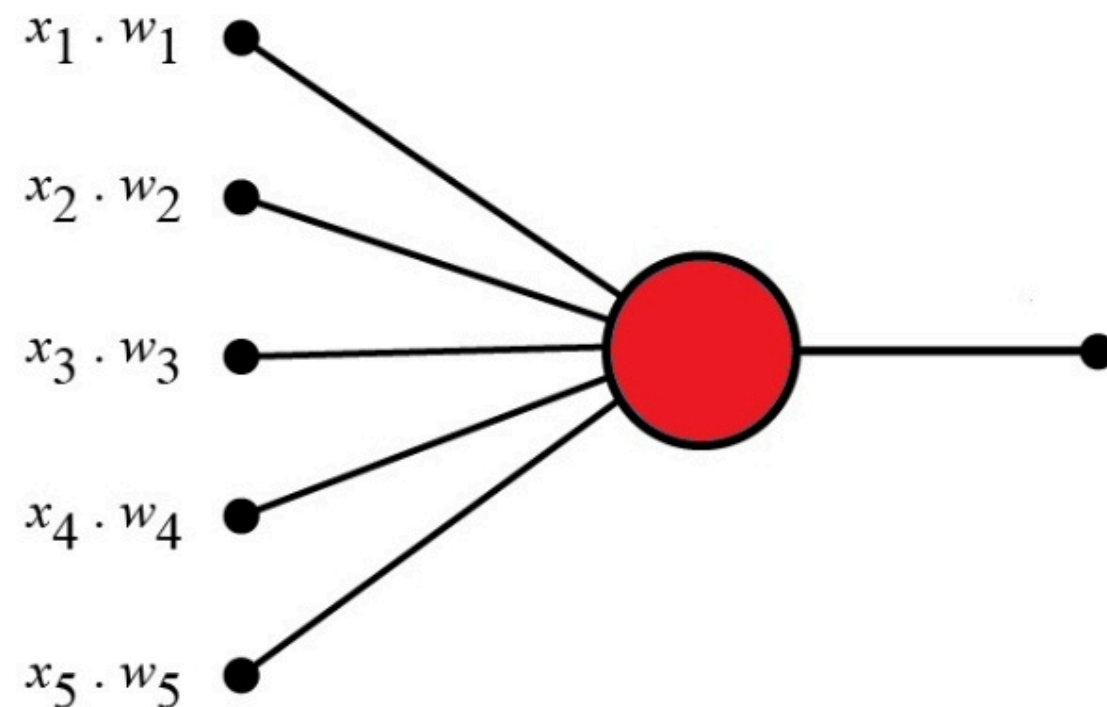
It is the simplest possible **Neural Network**.

Neural Networks are the building blocks of **Machine Learning**.

The Perceptron

The original **Perceptron** was designed to take a number of **binary** inputs, and produce one **binary** output (0 or 1).

The idea was to use different **weights** to represent the importance of each **input**, and that the sum of the values should be greater than a **threshold** value before making a decision like **yes** or **no** (true or false) (0 or 1).



1 Step Function

$$\hat{y} = f(z) = \begin{cases} +1 & \text{if } z \geq 0 \\ -1 & \text{if } z < 0 \end{cases}$$

Where:

- $z = w \cdot x + b$ (weighted sum of inputs plus bias)
- Output is either +1 or -1 (binary classification)

Intuition

- The Perceptron only cares about which side of the decision boundary the input falls on
- It doesn't output probabilities — just a class label
- That's why it's called a hard threshold activation

What is a Perceptron?

A Perceptron is the simplest type of artificial neural network.

It was introduced by Frank Rosenblatt.

It is used for binary classification (two classes only):

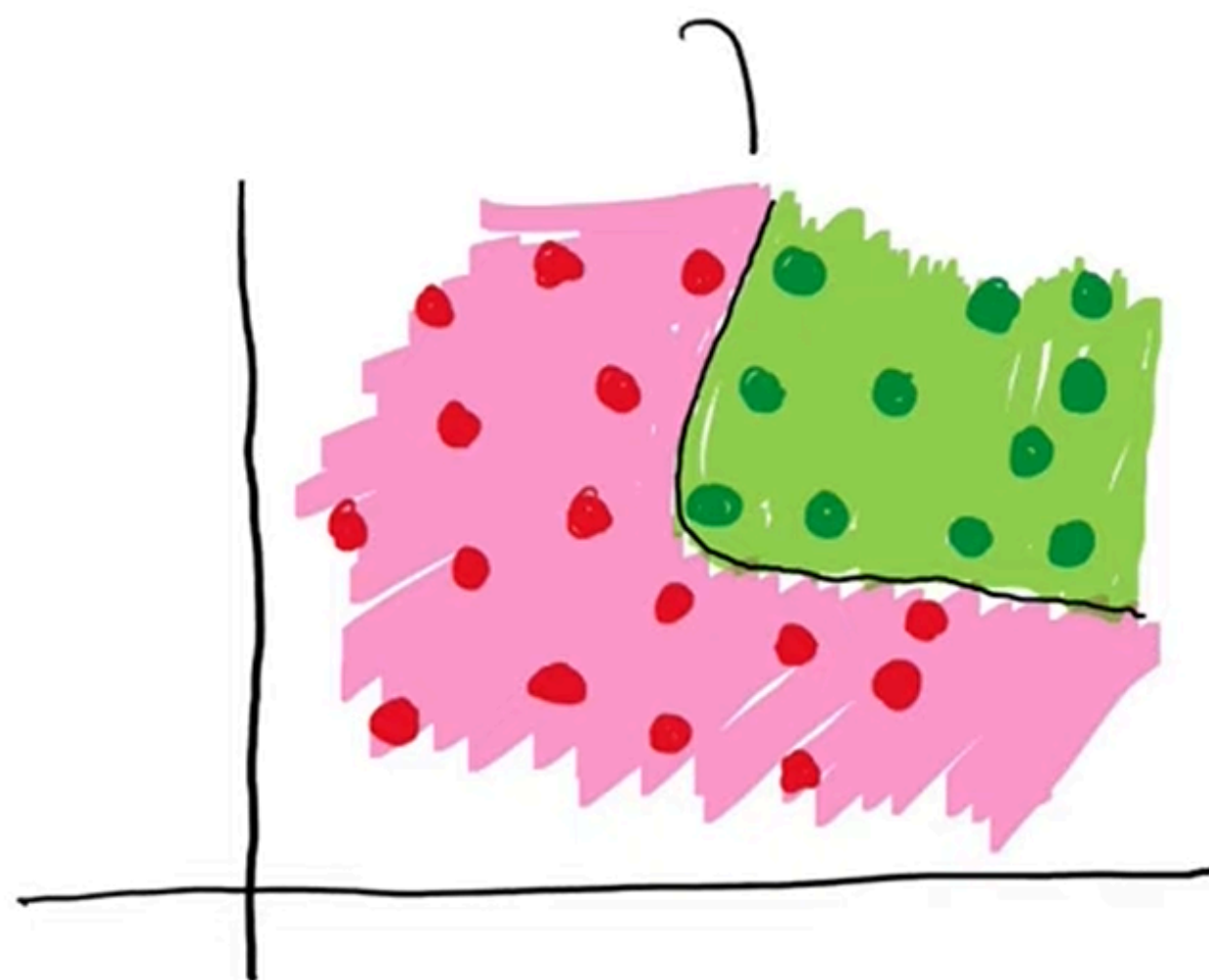
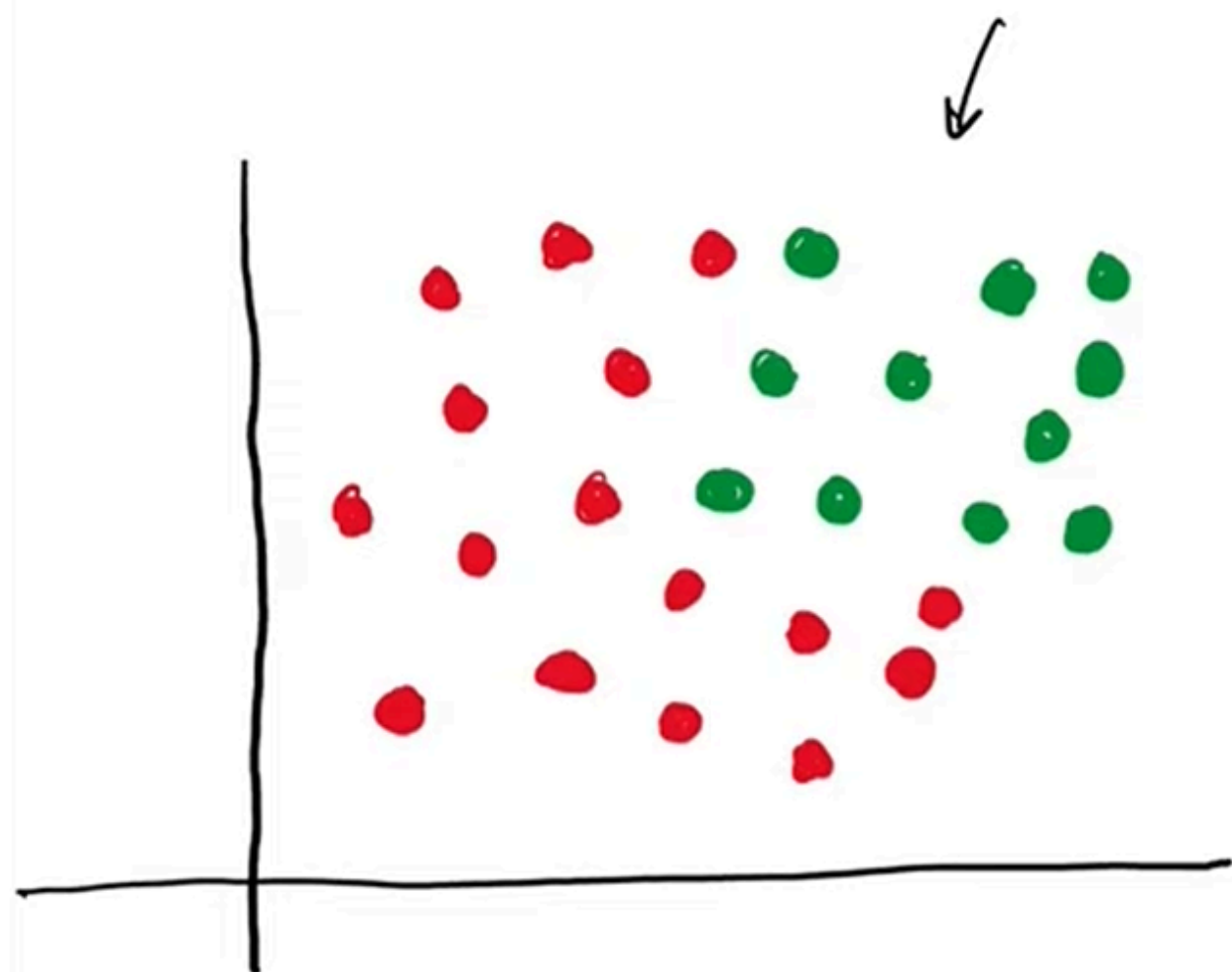
- 0 or 1
- Yes or No
- True or False

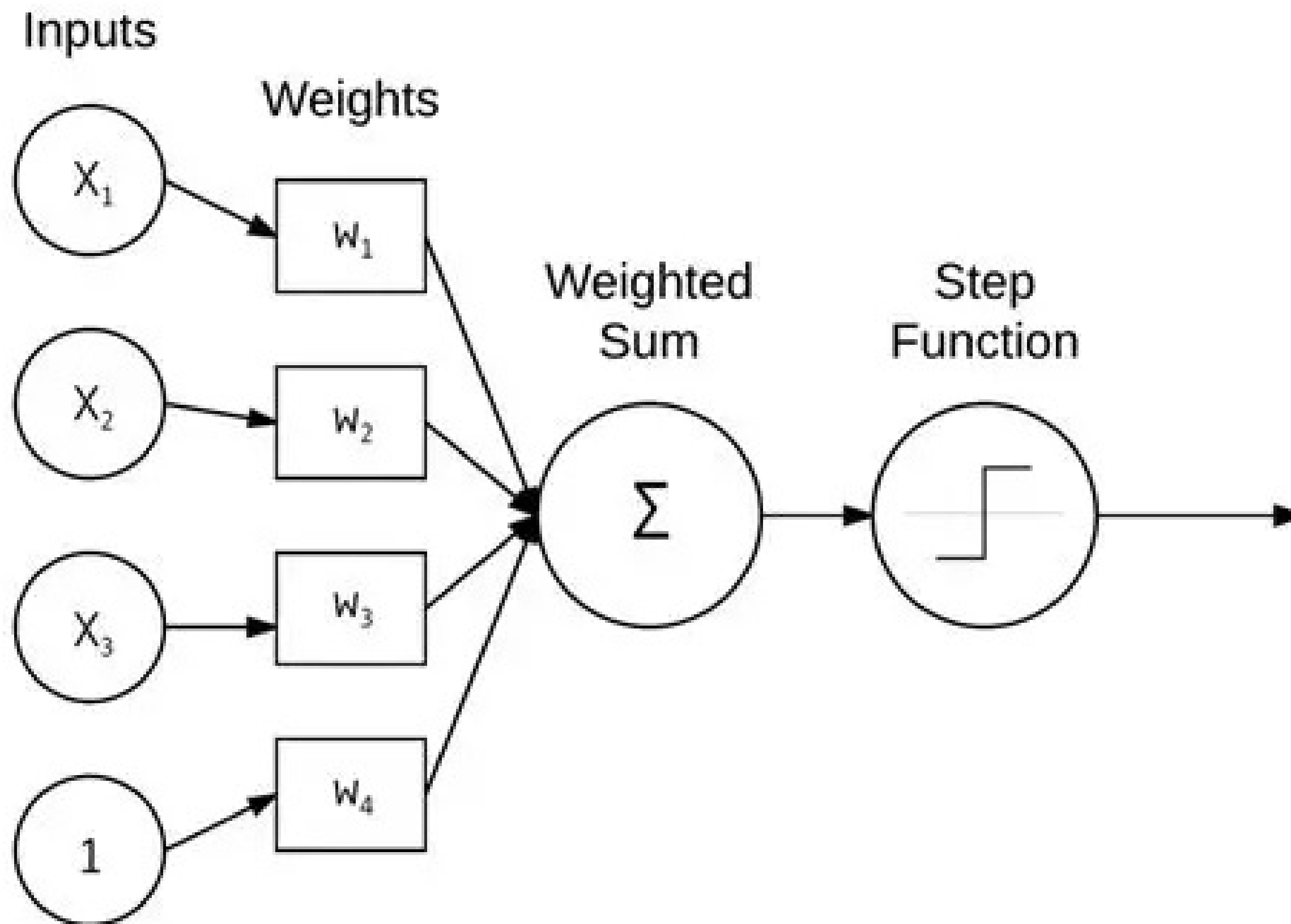
◆ Structure of a Perceptron

A perceptron has:

- Inputs $\rightarrow x_1, x_2, \dots, x_n$
- Weights $\rightarrow w_1, w_2, \dots, w_n$
- Bias $\rightarrow b$
- Activation function $\rightarrow$ Step function

check out logistic regressin perceptron is no difference with logistic regression traning
process





Training Data

Hours (x_1)	Attendance (x_2)	Output (y)
2	0	0
4	0	0
4	1	1
6	1	1

Step 1: Initialize

Let:

- $w_1 = 0$
- $w_2 = 0$
- Bias $b = 0$
- Learning rate $\alpha = 1$

Formula:

$$z = w_1x_1 + w_2x_2 + b$$

If $z \geq 0 \rightarrow 1$

Else $\rightarrow 0$

Training Process (1 Epoch)

◆ Example 1: $(2,0) \rightarrow y=0$

$$z = 0(2) + 0(0) + 0 = 0$$

Prediction = 1 (wrong ✖)

Update:

$$w = w + \alpha(y - \hat{y})x$$

$$w_1 = 0 + 1(0 - 1)(2) = -2$$

$$w_2 = 0 + 1(0 - 1)(0) = 0$$

$$b = 0 + 1(0 - 1) = -1$$

New weights:

- $w_1 = -2$
- $w_2 = 0$
- $b = -1$

◆ Example 2: $(4,0) \rightarrow y=0$

$$z = (-2)(4) + 0(0) - 1 = -9$$

Prediction = 0 

No update needed.

◆ Example 3: $(4,1) \rightarrow y=1$

$$z = (-2)(4) + 0(1) - 1 = -9$$

Prediction = 0 (wrong )

Update:

$$w_1 = -2 + 1(1 - 0)(4) = 2$$

$$w_2 = 0 + 1(1 - 0)(1) = 1$$

$$b = -1 + 1(1 - 0) = 0$$

New weights:

- $w_1 = 2$
- $w_2 = 1$
- $b = 0$

◆ Example 4: (6,1) → y=1

$$z = 2(6) + 1(1) + 0 = 13$$

Prediction = 1 ✓

No update needed.

🎯 Final Decision Boundary

$$2x_1 + 1x_2 + 0 = 0$$

The perceptron learned a line that separates pass and fail.

💡 Important Point

- ✓ Perceptron updates weights **only when wrong**
- ✓ Stops when all points are correctly classified (if linearly separable)

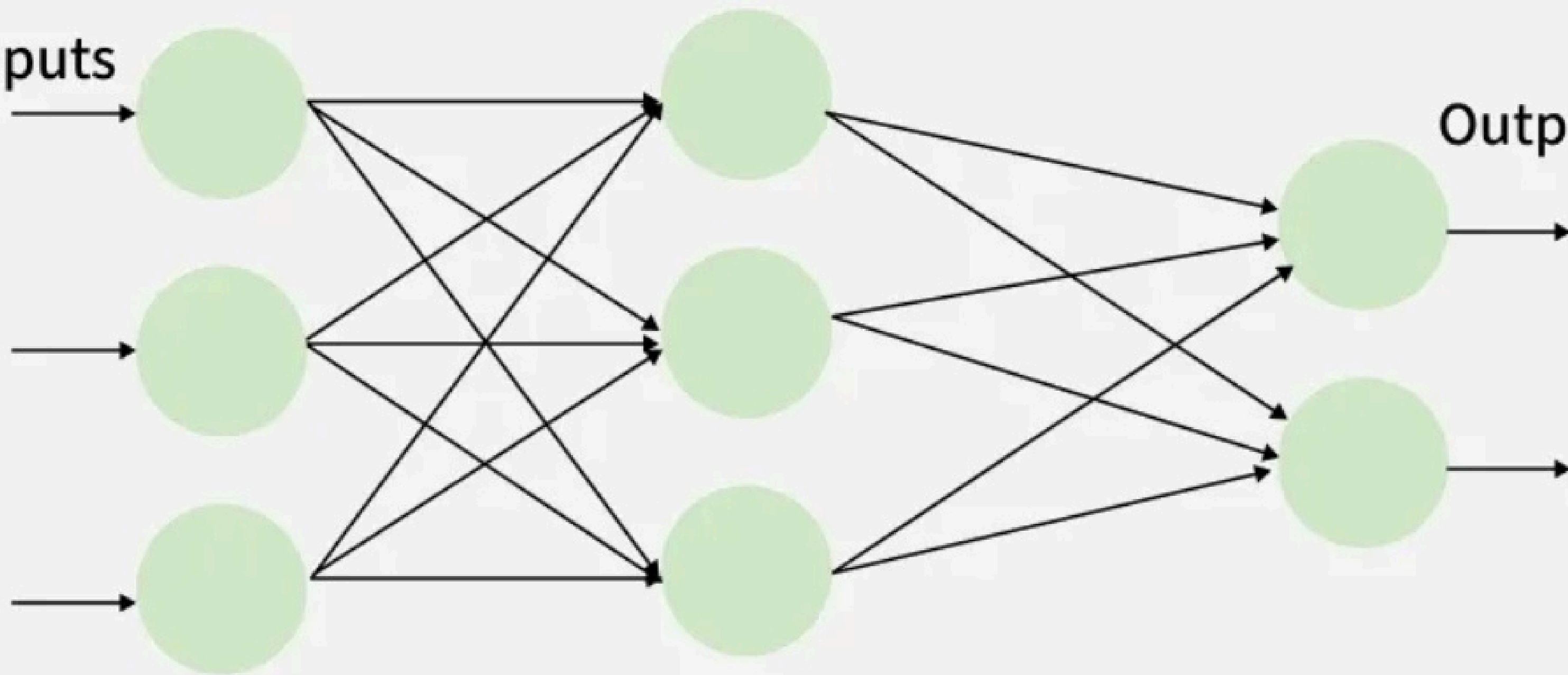
1 Step Function is Non-Differentiable

The Perceptron uses the **step (threshold)** activation:

$$\hat{y} = f(z) = \begin{cases} +1 & z \geq 0 \\ -1 & z < 0 \end{cases}$$

- This function **jumps abruptly** from -1 to +1.
 - It has **no slope** at the jump (discontinuous).
 - **Gradient descent requires derivatives** to know which direction to adjust weights.
- 💡 You can't take a derivative of a step function at the threshold → no gradient to follow.

inputs



Outputs

Input Layer

Hidden Layer

Output Layer

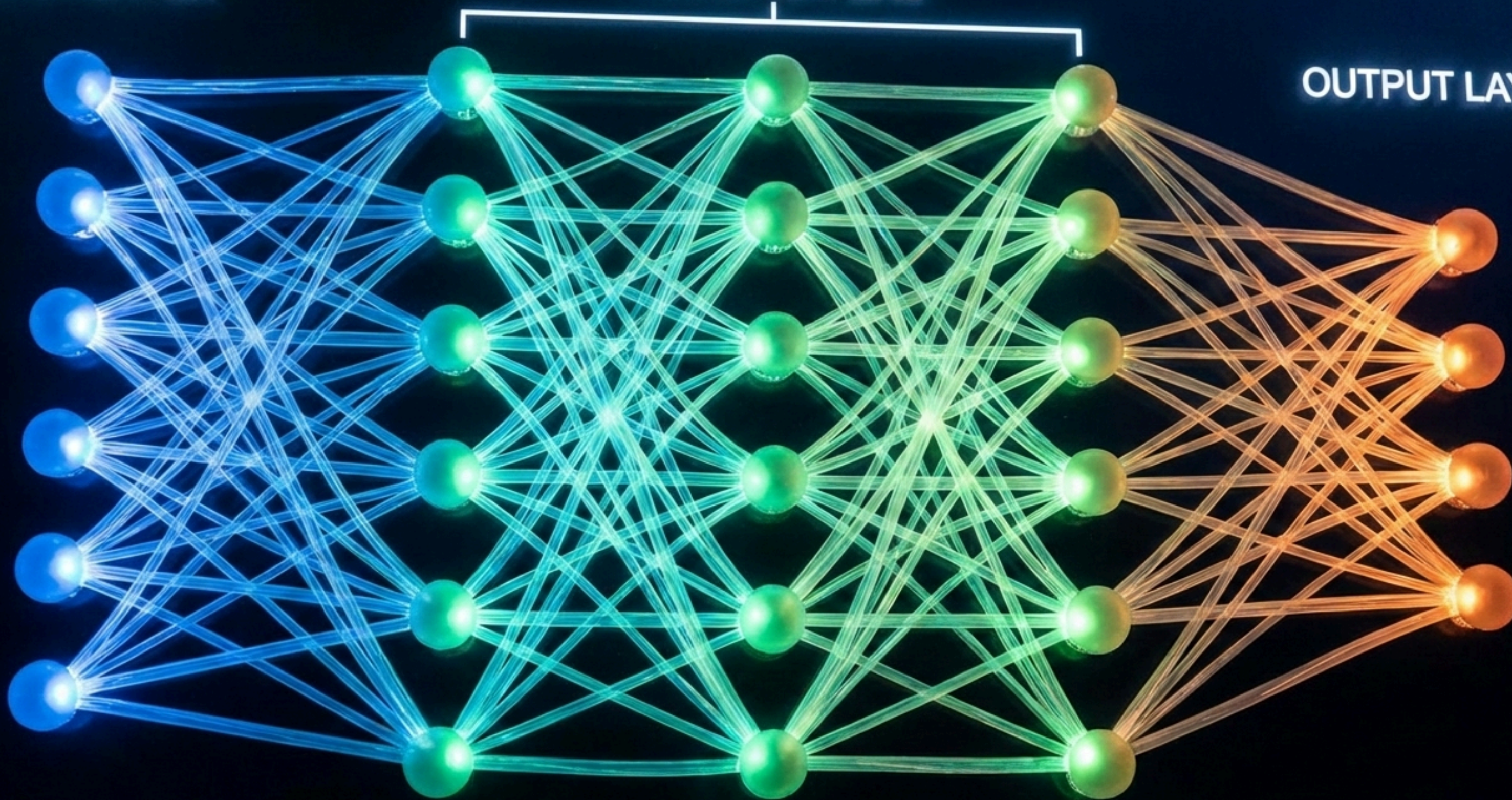
How MLP Solves This

- Add a hidden layer with non-linear activation.
- Now the network can combine multiple linear boundaries to create a non-linear boundary.

INPUT LAYER

HIDDEN LAYERS

OUTPUT LAYER

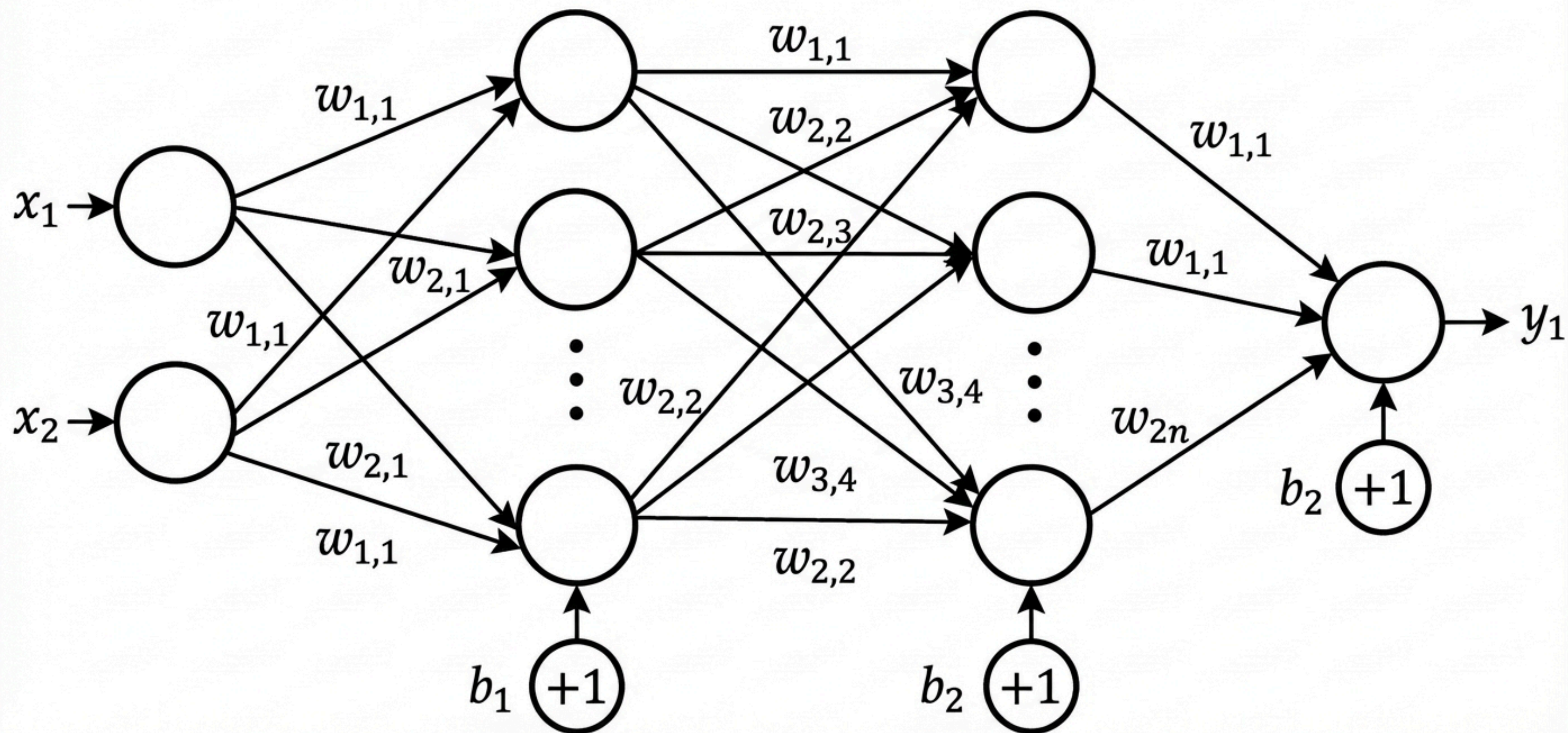


Input Layer

Hidden Layer 1

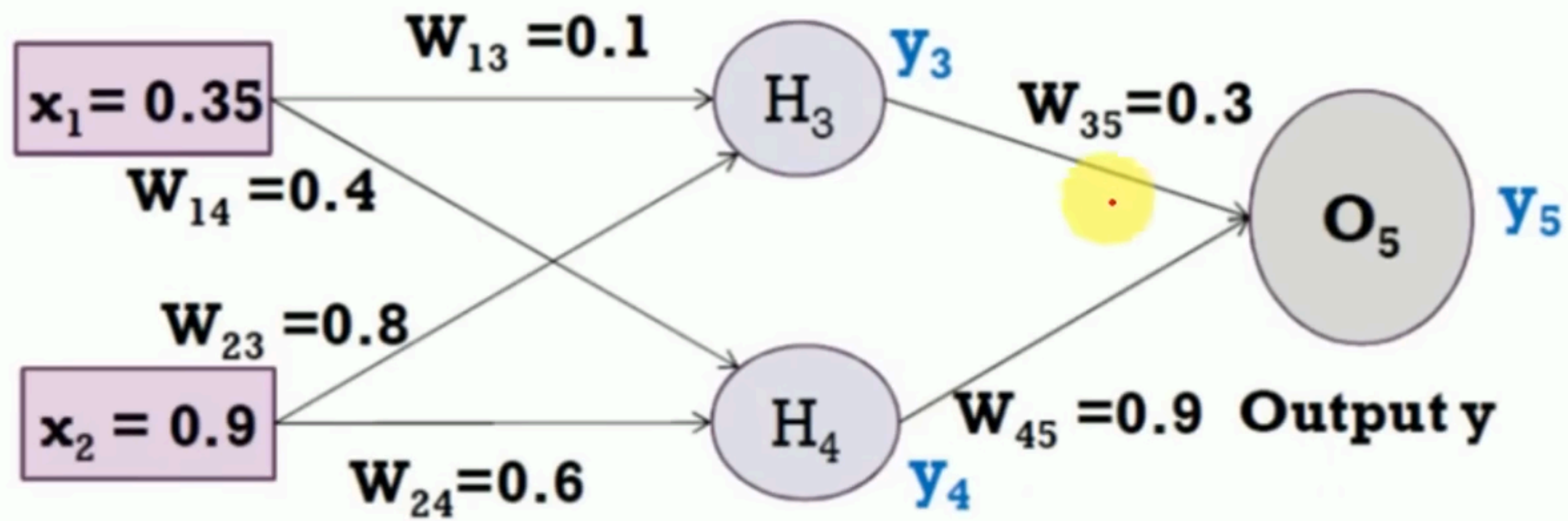
Hidden Layer 2

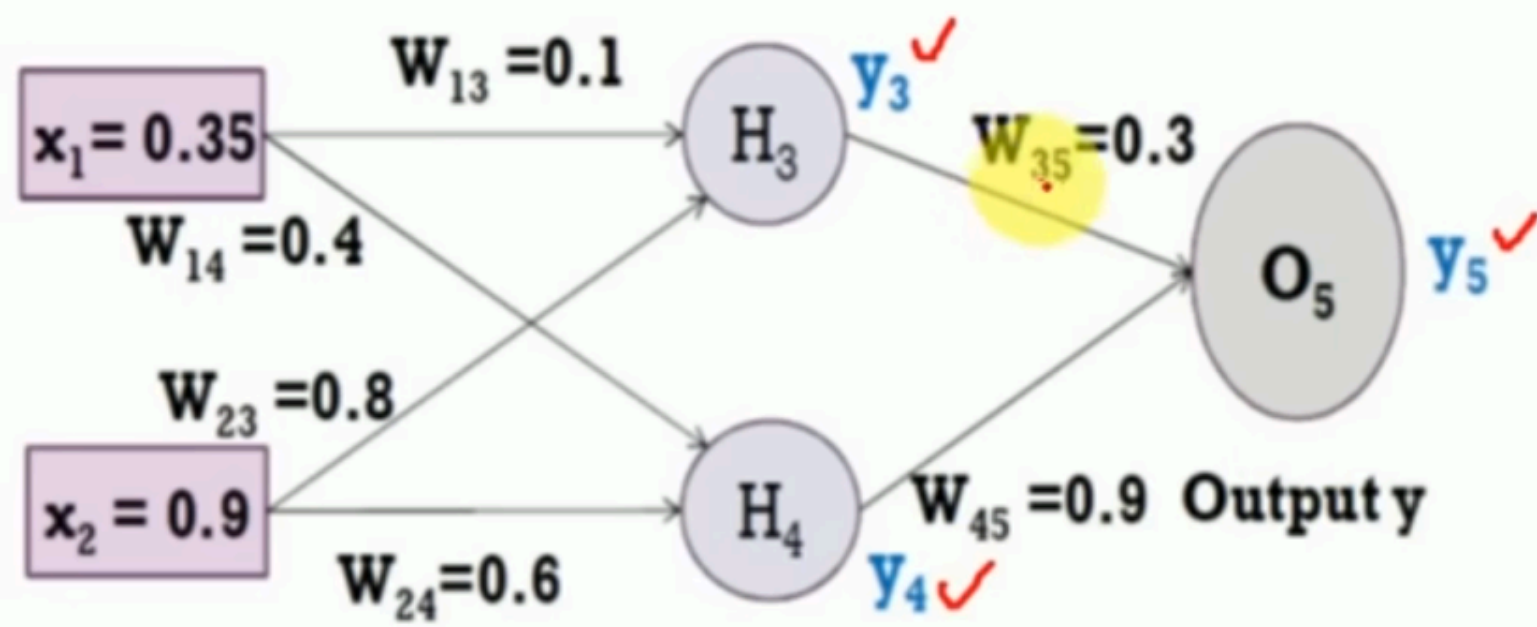
Output Layer



3 Hidden Layers vs Output Layer

Layer	Common Activation
Hidden	ReLU, Sigmoid, Tanh (non-linear)
Output (final)	Sigmoid → binary classification Softmax → multi-class classification Linear → regression





- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_j (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$a_1 = (w_{13} * x_1) + (w_{23} * x_2) \checkmark$$

$$= (0.1 * 0.35) + (0.8 * 0.9) = \underline{0.755}$$

$$y_3 = f(a_1) = 1 / (1 + e^{-0.755}) = \underline{0.68}$$

$$a_2 = (w_{14} * x_1) + (w_{24} * x_2) \checkmark$$

$$= (0.4 * 0.35) + (0.6 * 0.9) = \underline{0.68}$$

$$y_4 = f(a_2) = 1 / (1 + e^{-0.68}) = \underline{0.6637}$$

$$a_3 = (w_{35} * y_3) + (w_{45} * y_4)$$

$$= (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$$

$$y_5 = f(a_3) = 1 / (1 + e^{-0.801}) = \underline{0.69 \text{ (Network Output)}}$$